Warehouse Slotting Optimizer

Guide: Ms . Chandusha Asst.Prof.,CS&IT

2520090213-Ramya
2520080009-krishnasree

ABSTRACT

- Warehouse throughput depends significantly on **product placement**.
- Products frequently picked together should ideally be stored **close to each other**.
- Poor slot allocation increases **picker travel distance and order ful fillment time**.
- The proposed system analyzes **historical order and SKU data** to identify co-pick patterns.
- **Apriori / FP-Growth** is used to discover frequently co-picked products.
- Product relationships are converted into an **affinity matrix**.
- An optimization algorithm assigns products to suitable storage slots.
- The objective is to **minimize expected picking travel** while satisfying warehouse constraints.
- The optimized layout is evaluated against the existing layout.

INTRODUCTION

Background:
Warehouses store a large number of products across multiple storage locations.
Order picking is one of the most **time-consuming warehouse operations**.
A significant amount of picker time can be spent travelling between storage locations.
Traditional slotting often relies on **manual decisions or product demand alone**.

Problem:
Frequently co-picked products may be stored far apart.
This causes:

- Increased travel distance
- Longer picking time
- Higher labor effort
- Reduced warehouse throughput

Objective:
Analyze historical orders.
Discover product co-pick relationships.
Assign related products to nearby slots.
Minimize expected picker travel distance.

Software & Hardware Requirements:

Software

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas & NumPy
- Mlxtend — Apriori / FP-Growth
- Scikit-learn
- Matplotlib / Plotly
- Streamlit — Dashboard

Hardware

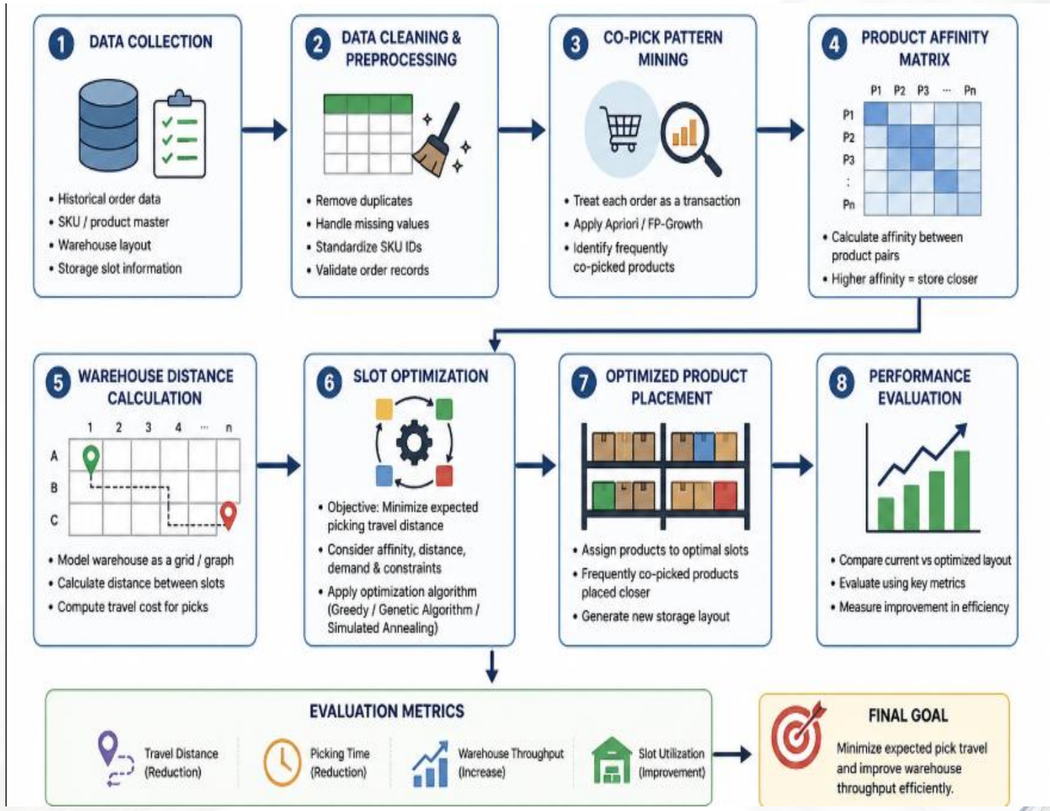
- Intel i3/i5 or equivalent
- 8 GB RAM or higher
- 20 GB storage
- Windows / Linux / macOS
- Internet connection

Literature survey:

Association Rule Mining: Apriori and FP-Growth identify frequently co-occurring products.
Warehouse Slotting: Determines suitable locations based on demand, accessibility, and product characteristics.
Order Picking: Research focuses on reducing picker travel and fulfillment time.
Optimization: Genetic Algorithms, Simulated Annealing, and Greedy methods solve complex slot-assignment problems.
Research Gap: Existing approaches often focus on individual product demand; our approach combines **co-pick affinity with slot distance**.

Methodology:

Data Collection → Order & SKU data
Data Cleaning → Remove errors & duplicates
Co-Pick Mining → Find frequently paired products
Affinity Analysis → Measure product relationships
Slot Optimization → Assign products to nearby slots
Evaluation → Measure travel time & throughput



THANK YOU